Machine Learning Experiment Guidebook

Experiment 2：Classification of Wine Dataset Using PCA and SVM Algorithms

1. Objective

The objective of this experiment is to employ Principal Component Analysis (PCA) as a dimensionality reduction technique in conjunction with the Support Vector Machine (SVM) algorithm to address the classification challenge presented by the Wine Dataset. By the end of this experiment, you should be able to understand the concept of PCA & SVM, implement the algorithm in Python, and evaluate the model's performance in classifying wine dataset.

2. Environment

- Operating System: Windows 10
- Python programming environment: Anaconda & Jupyter Notebook

3. Experiment Tasks

3.1 Install Anaconda and Jupyter Notebook

Note: The lab computers have been installed with Anaconda.
You should have already installed Anaconda and Jupyter Notebook in Experiment 1. If not, please refer to the Experiment 1 guide for installation.

Note: Please follow the below-listed steps to perform dimension reduction on the Wine Dataset using the PCA algorithm, and subsequently train a classifier with the SVM algorithm. Ensure to include screenshots of both the experimental code and its corresponding output results in your experiment report.

3.2 Load Wine Dataset

The wine dataset is from UCI Machine Learning Repository. It contains the results of a chemical analysis of wines grown in the same region in Italy but derived from three different cultivars. The analysis determined the quantities of 13 constituents found in each of the three types of wines.

This dataset consists of 178 samples, with each sample representing a different chemical analysis result. Each sample is described by 13 continuous attributes, representing the concentrations of various chemical compounds in the wines. The goal of the dataset is to classify the wines into one of the three cultivars based on these chemical measurements.

(1) Use pandas to load the wine dataset：

If you are not familiar with Pandas, you can refer to the following tutorial.
Getting started tutorials — pandas 2.2.2 documentation (pydata.org)

I have sent you the dataset. You can use it directly.

```
import pandas as pd
df_wine = pd.read_csv('wine.data', header=None)
```

Or you can download the dateset from the dateset link:
https://archive.ics.uci.edu/ml/machine-learning-databases/wine/wine.data

```
# Import the pandas library, which is used for data manipulation and analysis.
import pandas as pd

# Load the wine dataset from the UCI Machine Learning Repository
# header=None is specified because the file does not have column names in its first row.
df_wine = pd.read_csv('https://archive.ics.uci.edu/ml/machine-learning-databases/wine/wine.data', header=None)
```

(2) Assign meaningful column names to the DataFrame based on the dataset's features. Then display this dataset. Except for 'Class Label,' the remaining columns correspond to the attributes of the wine.

① Alcohol
② Malic acid
③ Ash
④ Alcalinity of ash
⑤ Magnesium
⑥ Total phenols
⑦ Flavanoids
⑧ Nonflavanoid phenols
⑨ Proanthocyanins
⑩ Color intensity
⑪ Hue
⑫ OD280/OD315 of diluted wines
⑬ Proline

```
# Assign meaningful column names to the DataFrame based on the dataset's features
df_wine.columns = [
```

```
        'Class label', 'Alcohol', 'Malic acid', 'Ash',
        'Alcalinity of ash', 'Magnesium', 'Total phenols',
        'Flavanoids', 'Nonflavanoid phenols', 'Proanthocyanins',
        'Color intensity', 'Hue',
        'OD280/OD315 of diluted wines', 'Proline'
]

# Display the first 5 rows of the DataFrame to verify correct data loading and formatting
df_wine.head()
```

	Class label	Alcohol	Malic acid	Ash	Alcalinity of ash	Magnesium	Total phenols	Flavanoids	Nonflavanoid phenols	Proanthocyanins	Color intensity	Hue	OD280/OD315 of diluted wines	Proline
0	1	14.23	1.71	2.43	15.6	127	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065
1	1	13.20	1.78	2.14	11.2	100	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050
2	1	13.16	2.36	2.67	18.6	101	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185
3	1	14.37	1.95	2.50	16.8	113	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480
4	1	13.24	2.59	2.87	21.0	118	2.80	2.69	0.39	1.82	4.32	1.04	2.93	735

(3) Splitting The Data into Training (70%) And Testing Dataset (30%).

```python
from sklearn.model_selection import train_test_split  # For splitting the dataset into training and testing sets

# Split the dataset into features (X) and target labels (y)
X = df_wine.iloc[:,1:].values  # Features
y = df_wine.iloc[:,0].values # Target labels

# Define the desired test set size or the train-test split ratio
test_size = 0.3  # 30% of the dataset will be used for testing

# Use train_test_split function to divide the dataset into training and testing sets
# Shuffle the data before splitting if desired (default is True)
#fixed random state for reproducibility
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=test_size, random_state=48)

# Print the sizes of the training and testing sets
print(f"Training set size: {len(X_train)} samples")
print(f"Test set size: {len(X_test)} samples")
```

```
Training set size: 124 samples
Test set size: 54 samples
```

3.3 PCA Dimensionality Reduction

Principal Component Analysis (PCA) is a dimensionality reduction technique widely used in machine learning and statistics to simplify high-dimensional datasets while retaining as much of the data's variability as possible. The process of applying PCA for dimensionality reduction involves several key steps:

(1) Data Preparation: Before starting PCA, it's essential to preprocess the data. This typically includes standardizing the features to have zero mean and unit variance, which is crucial for PCA since it is sensitive to the scale of variables.

- ✓ StandardScaler is a preprocessing technique commonly used in machine learning for feature scaling. It transforms the features of the input dataset by performing a standardization operation, ensuring that the distribution of each feature has a mean of 0 and a standard deviation of 1, thereby bringing all features to the same scale.

In Python's Scikit-learn library, you can use StandardScaler as follows:

```
# Import the StandardScaler module from the sklearn.preprocessing library,
#which is used for standardizing features by removing the mean and scaling to unit variance.
from sklearn.preprocessing import StandardScaler

# Instantiate the StandardScaler object. This object will be used to perform the standardization
#process on our datasets.
scaler = StandardScaler()

# Fit the StandardScaler to the training dataset (X_train).
#This step calculates the mean and standard deviation of the training data.
scaler.fit(X_train)

# Transform the training dataset using the fitted scaler.
#This applies the standardization, centering data around 0 and scaling to unit variance, to X_train.
X_train_std = scaler.transform(X_train)

# Transform the testing dataset using the same scaler fitted on the training data.
#This ensures that the scaling applied to the test set matches that of the training set.
X_test_std = scaler.transform(X_test)
```

(2) Covariance Matrix Calculation: PCA begins by calculating the covariance matrix of the standardized data. The covariance matrix quantifies how each feature varies with respect to every other feature.

✓ Use the np.cov function to compute the covariance matrix. np.cov() is a function from NumPy that computes the covariance matrix of a set of variables, in this context, the features of your dataset. The covariance matrix shows how each variable (feature) varies with respect to every other variable.

```
import numpy as np

# Calculate the covariance matrix of the standardized training data
# Transpose to switch rows and columns since np.cov expects features as rows
cov_mat = np.cov(X_train_std.T)

cov_mat.shape
```

(3) Eigenvalue Decomposition: The next step involves performing eigenvalue decomposition on the covariance matrix. This mathematical process uncovers the eigenvectors and eigenvalues of the covariance matrix. Eigenvectors represent the directions of maximum variance in the data, while eigenvalues quantify the magnitude of this variance in their respective directions.

✓ Use the numpy.linalg.eig function to obtain eigenvalues and eigenvectors. numpy.linalg.eig() is a function provided by the NumPy library for computing the eigenvalues and eigenvectors of a square matrix. Eigenvalues and eigenvectors are fundamental concepts in linear algebra.

```
# Compute eigenvalues and eigenvectors of the covariance matrix
eigen_vals, eigen_vecs = np.linalg.eig(cov_mat)

# Print the eigenvalues to see the variance explained by each principal component
print('\nEigenvalues \n%s' % eigen_vals)  # Displays the eigenvalues
```

```
Eigenvalues
[4.79238486 2.62792025 1.39308561 0.98592278 0.87537781 0.6705819
 0.48279911 0.09713136 0.31751144 0.29032342 0.23963852 0.15560255
 0.17741144]
```

✓ Next, we introduce the concept of **variance explained ratio**. The variance explained ratio of an eigenvalue is the proportion of this eigenvalue to the total sum of eigenvalues. While **Cumulative Variance Explained** is the sum of the variance explained ratios of each eigenvalue up to a certain point, indicating the total proportion of variance explained by retaining those eigenvalues.

✓ Calculate the individual variance explained (variance explained ratio) and the cumulative variance explained.

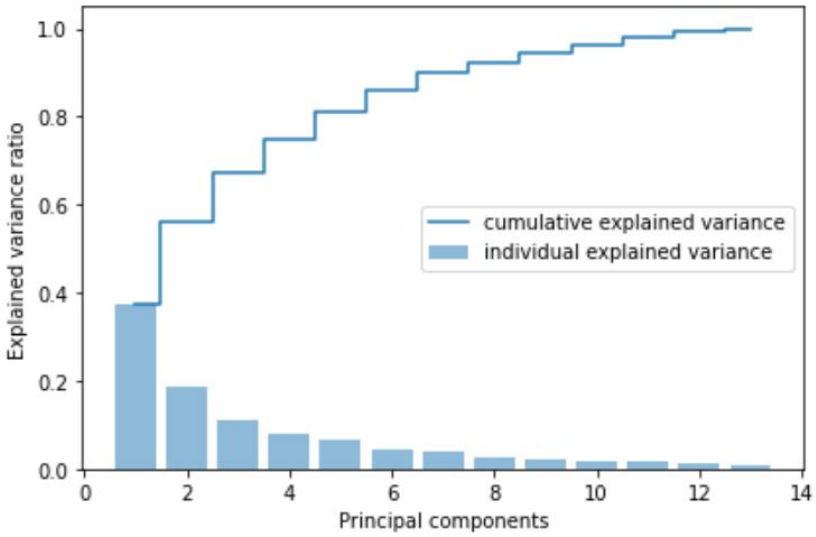
```
# Calculate the total of the eigenvalues
tot = sum(eigen_vals)

# Calculate the percentage of variance explained by each eigenvalue
# List comprehension to calculate variance explained, sorted by eigenvalue magnitude in descending order
var_exp = [(i / tot) for i in sorted(eigen_vals, reverse=True)]

# Calculate the cumulative variance explained
cum_var_exp = np.cumsum(var_exp)  # Cumulative sum of the explained variances
```

✓ Subsequently, plot the graph corresponding to the variance Individual Variance Explained and Cumulative Variance Explained.

```
import matplotlib.pyplot as plt
plt.bar(range(1, 14), var_exp, alpha=0.5, align='center',label='individual explained variance')
plt.step(range(1, 14), cum_var_exp, where='mid',label='cumulative explained variance')
plt.ylabel('Explained variance ratio')
plt.xlabel('Principal components')
plt.legend(loc='best')
plt.tight_layout()
plt.show()
```

```
import matplotlib.pyplot as plt

# Import the necessary module for plotting from Matplotlib library
import matplotlib.pyplot as plt

# Create a bar plot for the individual explained variance of each principal component
# var_exp contains the individual explained variance for each principal component
plt.bar(range(1, 14), var_exp, alpha=0.5, align='center',
        label='individual explained variance')  # Label for the legend

# Plot a step line graph to represent the cumulative explained variance
# cum_var_exp holds the cumulative explained variance up to each principal component
plt.step(range(1, 14), cum_var_exp, where='mid',
         label='cumulative explained variance')  # Label for the legend

# Add labels to the axes
plt.ylabel('Explained variance ratio')  # Y-axis label indicates the proportion of variance explained
plt.xlabel('Principal components')  # X-axis label refers to the principal components being considered

# Add a legend to the plot for clarity
plt.legend(loc='best')  # Position the legend optimally

# Adjust the layout to ensure everything fits without overlap
plt.tight_layout()

# Display the plot
plt.show()  # Renders the plot on screen
```

The plot should be like below:



Observations:

From the plot above, we can see that the first principal component accounts for nearly 40% of the variance (information), and the first two principal components together account for 60% of the variance.

(4) Selecting Principal Components: The eigenvectors associated with the largest eigenvalues are chosen as the principal components. These components form a new coordinate system that captures the most significant modes of variation in the data. Typically, one selects a number of components that explain a substantial portion of the total variance to balance between dimensionality reduction and information retention.

✓ Sort the eigenvalues:

```
eigen_pairs = [(np.abs(eigen_vals[i]), eigen_vecs[:, i]) for i in range(len(eigen_vals))]
eigen_pairs.sort(key=lambda k: k[0], reverse=True)
eigen_pairs
```

```
# Construct a list of tuples, each containing an eigenvalue and its corresponding eigenvector.
# np.abs(eigen_vals[i]) computes the absolute value of the i-th eigenvalue to handle any potential sign inconsistency.
# eigen_vecs[:, i] extracts the i-th column vector (eigenvector) from the eigen_vecs matrix.
# This loop iterates over the range of indices equivalent to the number of eigenvalues, creating a pair for each eigenvalue-vector combo.
eigen_pairs = [(np.abs(eigen_vals[i]), eigen_vecs[:, i]) for i in range(len(eigen_vals))]

# Sort the list of eigenvalue-eigenvector pairs based on the first element of each tuple (the eigenvalue).
# The key parameter in sort function specifies a function of one argument that is used to extract a comparison key from each input element.
# Here, we use a lambda function lambda k: k[0] which returns the first element of the tuple (the eigenvalue).
# Setting reverse=True ensures the sorting is done in descending order, meaning the tuple with the highest eigenvalue comes first.
eigen_pairs.sort(key=lambda k: k[0], reverse=True)

eigen_pairs
```

```
[(4.792384857696383,
  array([ 0.15665089, -0.25751677,  0.00524937, -0.24804323,  0.15272478,
          0.38029125,  0.42058583, -0.32160848,  0.29554616, -0.05912778,
          0.28682891,  0.37307024,  0.2950017 ])),
 (2.627920246888887,
  array([ 4.85795595e-01,  2.31195791e-01,  3.91112205e-01,  7.45554344e-02,
          2.35643992e-01,  6.98178486e-02, -2.67703410e-04,  8.73711553e-02,
          5.01268570e-02,  5.04024398e-01, -2.79409319e-01, -1.26204380e-01,
          3.65290369e-01])),
 (1.3930856138208747,
  array([ 0.18237532, -0.11797145, -0.52694618, -0.63714949, -0.00244807,
         -0.1677283 , -0.16287334, -0.17619201, -0.22169645,  0.23202655,
         -0.10088041, -0.20929948,  0.16282302])),
 (0.9859227826250783,
  array([-0.1070652 , -0.29241377,  0.23777491, -0.027828  ,  0.69606588,
         -0.26209408, -0.1850799 ,  0.06261639, -0.21012081, -0.19195245,
          0.34037929, -0.18250626,  0.15550055])),
```

✓ Constructs a matrix w by combining the two most important eigenvectors:

'eigen_pairs' is a list where each element is a tuple containing an eigenvalue at index 0 and its corresponding eigenvector at index 1.

```
# Here, we select the eigenvectors corresponding to the two largest eigenvalues:
# - eigen_pairs[0][1] fetches the eigenvector for the largest eigenvalue.
# - eigen_pairs[1][1] fetches the eigenvector for the second largest eigenvalue.
# The np.newaxis operation is applied to each selected eigenvector (using [:, np.newaxis]) to convert it
# from a 1D array to a 2D column
# vector, making it ready for horizontal stacking.
# np.hstack is then used to concatenate these two column vectors side by side, forming the matrix 'w'.

w = np.hstack((eigen_pairs[0][1][:, np.newaxis], eigen_pairs[1][1][:, np.newaxis]))

# displays the constructed matrix 'w'
print('Matrix W:\n', w)
```

```
Matrix W:
 [[ 1.56650889e-01  4.85795595e-01]
 [-2.57516773e-01  2.31195791e-01]
 [ 5.24936728e-03  3.91112205e-01]
 [-2.48043233e-01  7.45554344e-02]
 [ 1.52724776e-01  2.35643992e-01]
 [ 3.80291249e-01  6.98178486e-02]
 [ 4.20585828e-01 -2.67703410e-04]
 [-3.21608484e-01  8.73711553e-02]
 [ 2.95546162e-01  5.01268570e-02]
 [-5.91277839e-02  5.04024398e-01]
 [ 2.86828914e-01 -2.79409319e-01]
 [ 3.73070237e-01 -1.26204380e-01]
 [ 2.95001704e-01  3.65290369e-01]]
```

(5) Transforming Data: Once the principal components are identified, the original data is projected onto this new coordinate system. Each data point is represented as a linear combination of these components, effectively reducing the number of

dimensions. This projection is done by multiplying the standardized data matrix by the matrix of selected eigenvectors (principal components).

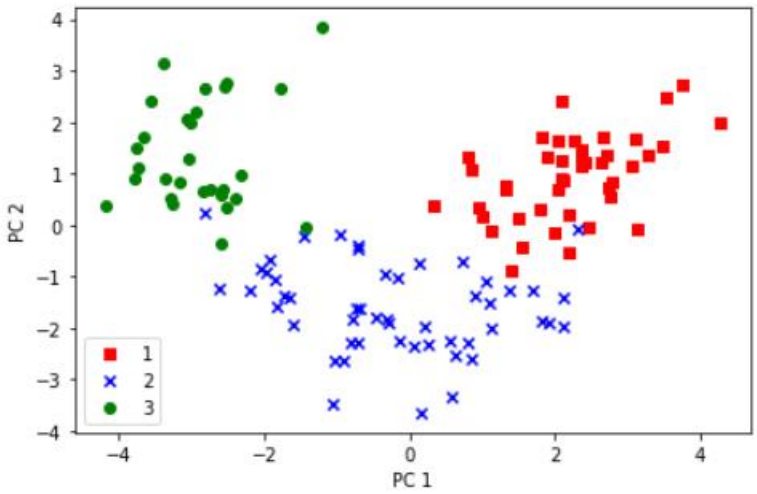
```
# Apply the transformation defined by the matrix 'w' onto the standardized training dataset 'train_data_std'.
# Here, 'dot' represents the dot product operation, which, in the context of matrices, performs a matrix multiplication.
# By multiplying the standardized training dataset with 'w', we project the original high-dimensional data onto a new feature space
# spanned by the principal components captured by 'w'.
# The resulting 'train_data_pca' is the transformed training dataset in this reduced-dimensional space.

X_train_pca = X_train_std.dot(w)

# Similarly, apply the same transformation 'w' to the standardized testing dataset 'test_data_std'.
X_test_pca = X_test_std.dot(w)

X_train_pca.shape
```

```
(124, 2)
```

(6) Visualization : In cases where the dataset is initially too complex for visualization due to high dimensions, the reduced data can now be plotted in 2D or 3D space, providing an intuitive visual representation of the structure within the data.

```
import matplotlib.pyplot as plt
colors = ['r', 'b', 'g']
markers = ['s', 'x', 'o']
for l, c, m in zip(np.unique(y_train), colors, markers):
      plt.scatter(X_train_pca[y_train == l, 0],
X_train_pca[y_train == l, 1],
c=c, label=l, marker=m)
plt.xlabel('PC 1')
plt.ylabel('PC 2')
plt.legend(loc='lower left')
plt.tight_layout()
plt.show()
```

```
# Import the necessary module for plotting from Matplotlib library
import matplotlib.pyplot as plt

# Define color and marker styles for different classes in the data
colors = ['r', 'b', 'g']  # Red, Blue, Green for class distinction
markers = ['s', 'x', 'o']  # Square, Cross, Circle for class distinction

# Iterate over unique class labels, colors, and markers simultaneously
for label, color, marker in zip(np.unique(y_train), colors, markers):
    # Select data points for the current class label from the PCA-transformed training data
    # X_train_pca[y_train == l, 0] picks PC1 values, X_train_pca[y_train == l, 1] picks PC2 values
    plt.scatter(X_train_pca[y_train == label, 0],
                X_train_pca[y_train == label, 1],
                c=color,  # Use the corresponding color for the class
                label=label,  # Label the class for the legend
                marker=marker)  # Use the corresponding marker for the class

# Set the label for the x-axis to 'PC 1', representing the first principal component
plt.xlabel('PC 1')
# Set the label for the y-axis to 'PC 2', representing the second principal component
plt.ylabel('PC 2')

# Place the legend in the lower left corner of the plot for clarity
plt.legend(loc='lower left')
# Adjust the plot to ensure all elements fit without overlapping
plt.tight_layout()
# Display the plot showing the scatter plot of PCA-transformed data colored and marked by class
plt.show()
```



From the scatter plot, we can observe that even after reducing the data to two dimensions using PCA, there still exist significant differences between data points from different classes. We can use a linear decision boundary to classify them.

By above steps, PCA reduces the complexity of the dataset, making it easier to visualize, analyze, and often improving the performance of machine learning algorithms by eliminating noise and redundant features.

3.4 Generating Model

Let's build support vector machine model. First, import the SVM module and create support vector classifier object by passing argument kernel as the linear kernel in SVC() function. Then, fit your model on train set using fit() and perform prediction on the test set using predict().

```
# Importing necessary modules from scikit-learn
from sklearn import svm
from sklearn import metrics
import time # Importing the time module to measure execution time

# Section: Training and Evaluating SVM Without PCA
start_time = time.time()
# Initialize the Support Vector Classifier with a linear kernel
svm_no_pca = svm.SVC(kernel='linear')
# Train the model using the original training data (X_train) and corresponding labels (y_train)
svm_no_pca.fit(X_train, y_train)
# Make predictions on the test dataset (X_test)
y_pred_no_pca = svm_no_pca.predict(X_test)
# Calculate the elapsed time for training and prediction without PCA
no_pca_time = time.time() - start_time
# Print the accuracy of the model trained without PCA
print("Without PCA - Accuracy:")
print(metrics.accuracy_score(y_test, y_pred_no_pca))
```

```python
# Section: Training and Evaluating SVM With PCA
start_time = time.time()
# Initialize another Support Vector Classifier instance
svm_pca = svm.SVC(kernel='linear')
# This time, train the model using the PCA-transformed training data (X_train_pca)
svm_pca.fit(X_train_pca, y_train)
# Predict on the PCA-transformed test dataset (X_test_pca)
y_pred_pca = svm_pca.predict(X_test_pca)
pca_time = time.time() - start_time
# Print the accuracy of the model trained with PCA
print("\nWith PCA - Accuracy:")
print(metrics.accuracy_score(y_test, y_pred_pca))

# Compare and print the execution times
print("\nTime Comparison:")
print(f"No PCA: {no_pca_time} seconds, PCA: {pca_time} seconds")
```

```
Without PCA - Accuracy:
0.9444444444444444

With PCA - Accuracy:
0.9444444444444444

Time Comparison:
No PCA: 0.12590265274047852 seconds, PCA: 0.00208282470703125 seconds
```

Observations: What do you discover from the results?

3.5 PCA in Scikit-learn

Scikit-learn provides a straightforward implementation of PCA through the class “sklearn.decomposition.PCA” . It uses a very similar process to other preprocessing functions that come with Scikit-learn.

Initialize the PCA object, specifying the number of components you want to keep (if you want to transform the data to have only two dimensions while retaining as much information as possible, you can set n_components as 2). Then, fit and transform your data.

```python
# Section: Training and Evaluating SVM With PCA in Scikit-Learn
# Import the PCA class from the decomposition module of the scikit-learn library
from sklearn.decomposition import PCA

# Initialize PCA for dimensionality reduction, keeping only 2 principal components
pca = PCA(n_components=2)

# Apply PCA transformation to standardized training dataset
X_train_pca = pca.fit_transform(X_train_std)

# Apply the same PCA transformation to standardized testing dataset (only transform)
X_test_pca = pca.transform(X_test_std)

# Initialize a Support Vector Machine (SVM) classifier with linear kernel, regularization parameter C=1,
# and a random state for reproducibility
clf = svm.SVC(kernel='linear', C=1, random_state=42)

# Train the SVM classifier using the transformed training dataset
clf.fit(X_train_pca, y_train)

# Predict labels for the transformed testing dataset
y_pred = clf.predict(X_test_pca)

# Calculate and print the accuracy score comparing predicted labels with actual labels of the testing dataset
print(metrics.accuracy_score(y_test, y_pred))
```

```
0.9444444444444444
```

The following code calculates the variance proportion explained by each principal component after PCA dimensionality reduction and prints them out.

```python
# Import the PCA class from the decomposition module of the scikit-learn library
from sklearn.decomposition import PCA
# Initialize the PCA object with 13 (all) as the number of principal components to be kept
pca = PCA(n_components=13)
# Fit the PCA model to the standardized training dataset (X_train_std) and transform it
X_pca = pca.fit_transform(X_train_std)

# Compute the ratio of the explained variance of each principal component
explained_variance = pca.explained_variance_ratio_
# Print the explained variance as a percentage for each principal component
print("Explained Variance by Principal Components:")
# Enumerate is used to iterate over explained_variance with an index starting from 1
for i, var in enumerate(explained_variance, start=1):
    # Format the output to display the index of the PC and its explained variance in percentage
    print(f"PC {i}: {var*100:.2f}%")
```

```
Explained Variance by Principal Components:
PC 1: 36.57%
PC 2: 20.05%
PC 3: 10.63%
PC 4: 7.52%
PC 5: 6.68%
PC 6: 5.12%
PC 7: 3.68%
PC 8: 2.42%
PC 9: 2.22%
PC 10: 1.83%
PC 11: 1.35%
PC 12: 1.19%
PC 13: 0.74%
```

To visualize these data, we can use these variance proportions to plot a bar chart with two Y-axes: one showing the variance proportion explained by each individual principal component, and the other (as a red line graph) showing the cumulative variance explained, helping us visually determine how many principal components to select in order to achieve a satisfactory variance retention effect.

```
import matplotlib.pyplot as plt
cumulative_explained_variance = np.cumsum(explained_variance)
plt.figure(figsize=(10, 6))
plt.bar(range(1, len(explained_variance)+1), explained_variance, tick_label=[f'PC {i}' for i
in range(1, len(explained_variance)+1)])
plt.ylabel('Explained Variance Ratio (%)')
plt.xlabel('Principal Component')

plt.twinx()
plt.plot(range(1, len(cumulative_explained_variance)+1), cumulative_explained_variance*100,
color='r', marker='o', linestyle='--')
plt.ylabel('Cumulative Explained Variance (%)')
plt.title('Explained Variance per Principal Component')
plt.show()
```

```python
# Import the necessary library for plotting
import matplotlib.pyplot as plt

# Calculate the cumulative explained variance using numpy's cumsum function
cumulative_explained_variance = np.cumsum(explained_variance)

# Create a new figure with specified size
plt.figure(figsize=(10, 6))

# Draw a bar chart to visualize individual explained variance ratios for each principal component
# The x-axis represents the principal components (labeled PC 1, PC 2, etc.)
# The y-axis shows the percentage of variance explained by each component
plt.bar(range(1, len(explained_variance)+1), explained_variance,
        tick_label=[f'PC {i}' for i in range(1, len(explained_variance)+1)])
plt.ylabel('Explained Variance Ratio (%)')
plt.xlabel('Principal Component')

# Add another y-axis to the plot for cumulative explained variance
plt.twinx()

# Plot the cumulative explained variance as a line chart with markers and a dashed line style
plt.plot(range(1, len(cumulative_explained_variance)+1), cumulative_explained_variance*100,
         color='r', marker='o', linestyle='--')
plt.ylabel('Cumulative Explained Variance (%)')

# Set the title of the plot
plt.title('Explained Variance per Principal Component')

# Display the plot
plt.show()
```